

Programowanie Równoległe na Architekturach Wielordzeniowych

Parallel programming for multi-core architectures

2025/2026

Parallel implementation evaluation report

1. Project details:

Project title:	Matrix multiplication
Student's name:	Adam Cherek
Index number:	s207250
Email:	s207250@student.pg.edu.pl

2. Description of data used for experiments:

Two Matrices: A and B of the same sizes, filled with random numbers (using same seed for reproducibility). For comparison between solutions of the same architecture (CPU-OpenMP or GPU-CUDA), Matrices 1024x1024 were used. For comparison of optimized solutions between architectures Matrices 8192x8192 and 16x16 were used.

Code used for filling matrices:

```
for (int i = 0; i < rows * cols; i++)  
{  
    matrix[i] = (float)(rand() % 100) / 10.0f;  
}
```

with srand set to 42 for reproducibility required to compare both solutions.

3. Environment #1 - OpenMP:

OS: Windows 10 with WSL 2.0 (Ubuntu-24.04)

CPU: 13th Gen Intel(R) Core(TM) i5-13500 (20 CPUs), ~2.5GHz

OpenMP: 201511 = v4.5

4. Environment #2 - CUDA:

OS: Windows 10 with WSL 2.0 (Ubuntu-24.04)

GPU: NVIDIA GeForce RTX 5060 Ti

CUDA: release 12.0, V12.0.140

5. Test results:

CUDA (1024x1024)						
Single Thread		Unoptimized		Optimized		
Computation	Total	Computation	Total	Computation	Total	
28.075184	28.781157	0.002496	0.364943	0.001875	0.349609	
28.044498	29.132320	0.002424	0.327172	0.002049	0.356505	
27.882488	29.080228	0.002292	0.371278	0.002157	0.347203	
28.240824	29.317239	0.002677	0.346540	0.001904	0.329427	
27.763258	28.860307	0.002247	0.366967	0.002108	0.437874	
27.689184	28.814218	0.002365	0.371365	0.001856	0.363448	
28.670685	29.772909	0.002267	0.357251	0.002440	0.384546	
28.953054	30.129612	0.002269	0.366992	0.002131	0.346229	
27.699165	28.846024	0.002307	0.358907	0.001858	0.364256	
28.625839	29.789395	0.002412	0.362399	0.001783	0.376573	
Average (s)	28.164418	29.252341	0.002376	0.359381	0.002016	0.365567
Speed Up (x)	-	-	11855.707	81.396	1.178	0.983
Uncertainty (s)	0.631935	0.6742275	0.000215	0.0220965	0.0003285	0.0542235

OpenMP (1024x1024)						
Single Thread		Unoptimized		Optimized		
Computation	Total	Computation	Total	Computation	Total	
2.522359	2.522363	0.235624	0.235633	0.056950	0.056953	
2.537815	2.537819	0.222654	0.222657	0.060655	0.060658	
2.520432	2.520437	0.232765	0.232769	0.052206	0.052209	
2.540680	2.540685	0.229678	0.229681	0.061448	0.061451	
2.539717	2.539721	0.245453	0.245456	0.044105	0.044108	
2.543741	2.543745	0.277158	0.277161	0.059834	0.059837	
2.557131	2.557136	0.224389	0.224392	0.059649	0.059653	
2.594872	2.594876	0.239986	0.239990	0.055545	0.055548	
2.654141	2.654145	0.248657	0.248661	0.057836	0.057840	
2.557421	2.557425	0.222326	0.222328	0.063270	0.063274	
Average (s)	2.556831	2.556835	0.237869	0.237873	0.057150	0.057153
Speed Up (x)	-	-	10.749	10.749	4.162	4.162
Uncertainty (s)	0.0668545	0.066854	0.027416	0.0274165	0.0095825	0.009583

CUDA VS OpenMP (8192x8192)				
CUDA Optimized		OpenMP Optimized		
Computation	Total	Computation	Total	
0.570773	1.139583	15.114283	15.114287	
0.542156	1.126889	14.440683	14.440686	
0.541396	1.111028	15.399648	15.399651	
0.541427	1.105994	14.956341	14.956346	
0.540996	1.158757	16.081724	16.081728	
0.612105	1.253726	15.910771	15.910775	
0.579283	1.202102	16.065123	16.065127	
0.569220	1.124283	15.565825	15.565829	
0.563609	1.120380	14.344358	14.344362	
0.541164	1.156704	15.040147	15.040152	
Average (s)	0.560213	1.149945	15.291890	15.291894
Speed Up (x)	27.297	13.298	0.037	0.075
Uncertainty (s)	0.0355545	0.073866	0.868683	0.868683

CUDA VS OpenMP (16x16)				
CUDA Optimized		OpenMP Optimized		
Computation	Total	Computation	Total	
0.000750	0.380957	0.034620	0.034621	
0.000841	0.384744	0.030240	0.030241	
0.000889	0.380302	0.034602	0.034603	
0.000731	0.370231	0.024834	0.024835	
0.000970	0.408231	0.042500	0.042500	
0.001285	0.389799	0.036623	0.036624	
0.000816	0.419834	0.022456	0.022457	
0.000876	0.371825	0.020348	0.020349	
0.000936	0.381057	0.029998	0.029999	
0.000729	0.424107	0.033790	0.033791	
Average (s)	0.000882	0.391109	0.031001	0.031002
Speed Up (x)	35.137	0.079	0.028	12.616
Uncertainty (s)	0.000278	0.026938	0.011076	0.0110755

6. Implementation #1 - OpenMP details:

SingleThread

This implementation uses OpenMP with a single thread (`omp_set_num_threads(1)`). Despite having the `#pragma omp parallel` for directive, only one thread executes the nested loops that compute all matrix elements. The parallelization overhead is present but unused, making this a baseline sequential version.

Unoptimized

This version parallelizes across all available CPU cores using `#pragma omp parallel` for `collapse(2)` with dynamic scheduling. Each thread computes one or more output elements (row-column pairs) independently by iterating through the entire N dimension. While threads execute in parallel, each thread repeatedly accesses scattered global memory locations, causing poor cache locality and frequent cache misses.

Optimized

This implementation applies cache-conscious tiled matrix multiplication with OpenMP parallelization. It divides matrices into 32×32 tiles and processes them in three nested loops (ii, jj, kk for tile indices). The outermost two tile loops are parallelized with `collapse(2)`, while the inner tile multiplication reuses loaded data across multiple operations. By working on smaller tile blocks that fit in L3 cache, memory bandwidth utilization improves significantly. The algorithm also reuses the value $a_val = A[i * N + k]$ within the j-loop to minimize repeated loads, further boosting performance compared to the unoptimized version.

7. Implementation #2 - CUDA details:

SingleThread

This implementation uses a single CUDA thread (1×1 grid) to perform the entire matrix multiplication sequentially. It executes three nested loops to compute all elements of the result matrix. While simple, this approach completely ignores GPU parallelism and will be extremely slow compared to multi-threaded approaches.

Unoptimized

This version distributes work across multiple threads organized in a grid of blocks (16×16 threads per block). Each thread independently computes one element of the result matrix by iterating through the entire N dimension. Threads access global memory directly for every operation, causing poor memory bandwidth utilization and excessive global memory latency without any optimization.

Optimized

This implementation uses tiled matrix multiplication with shared memory optimization. It divides matrices into 16×16 tiles and processes them iteratively, loading tiles into fast shared memory before computation. Each thread computes a partial dot product using cached tile data, then synchronizes with `__syncthreads()` to ensure correctness. Multiple threads cooperatively load data, reducing global memory traffic significantly. The approach maximizes memory bandwidth reuse and improves cache locality, resulting in substantially better performance than the unoptimized version while maintaining correctness.

8. Final notes and conclusions:

Even though the project seems not very spectacular, I've sunk into it deep enough to decide to enlarge a little base requirements. I've created 3 solutions (SingleThread, Unoptimized and Optimized) for both GPU Environment CUDA based solution and CPU Environment OpenMP based solution. Those are my conclusions from this project:

1. Performed experiment has clearly shown that Matrix Multiplication is a perfect task for parallelisation on GPU. Speed up of CUDA based implementation in comparison to OpenMP based implementation is growing along with the growth of the matrices size.
2. A single thread on CPU is worth much more than a single thread on GPU. The computation time difference between those two is enormous. This is also the reason why I've decided to compare only optimized solutions between environments, it would require too much time to compare single thread CUDA solution on bigger matrices. Long story short: CPU = Quality, GPU = Quantity.
3. Even though computation time on GPU (CUDA) was almost always much better than corresponding time on CPU (OpenMP), time required for copying results back from GPU is big enough to reconsider using CUDA based solution in some situations. For smaller matrices (like 16x16 visible on one of the attached tests results) use of CPU native solution might be highly beneficial, so even for problems perfect for GPU it is still recommended to think about in what real scenarios the program would be used.

9. Survey:

Fill the answers for questions related to frameworks that you used in your project:

1. How many lines of code did you write for:
 - a. OpenMP implementation: 124/125/140
 - b. CUDA/OpenCL implementation: 140/148/183
2. How would you describe programming difficulty of each framework/interface:
 - a. OpenMP: 5/10
 - b. CUDA/OpenCL: 7/10